Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

Implementación y análisis del Generador Congruencial Lineal en Python y Java

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emilio Josue Flores Gallardo
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Implementación y análisis del Generador Congruencial Lineal en Python y Java
Nombre del Docente	José O. Guamán Q.
Fecha	29 y 30 de octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: comprender el funcionamiento de un generador pseudoaleatorio mediante la implementación del método LCG.

Objetivos específicos:

- Implementar un generador LCG en Python.
- Implementar el mismo algoritmo en Java.
- Comparar las secuencias generadas y analizar sus propiedades.

3. Materiales, Reactivos, Equipos y Herramientas

Software y herramientas:

- **Python 3.12+ (`Matplotlib`):** implementación del LCG, análisis de Hull–Dobell y gráficas.
- **Java SE / JDK (`Swing`, `AWT`):** implementación equivalente con enteros de 64 bits (`long`) y gráficas nativas.

- **VS Code y terminal GNU/Linux:** edición, compilación y ejecución.
- **GitHub:** control de versiones y repositorio.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

El **Generador Congruencial Lineal (LCG)** calcula una secuencia determinista de enteros pseudoaleatorios y los normaliza en $[0, 1)$ mediante:

$$X_{n+1} = (aX_n + c) \bmod m, \qquad U_n = \frac{X_n}{m} \tag{1}$$

donde X_0 es la semilla, a el multiplicador, c el incremento y $m > 0$ el módulo. Según el **Teorema de Hull–Dobell**, un LCG mixto ($c > 0$) tiene período completo ($P = m$) si y solo si:

1. $\gcd(c, m) = 1$ (c y m son coprimos).
2. $(a - 1)$ es divisible por todo factor primo p de m.
3. Si $4 \mid m$, entonces $4 \mid (a - 1)$.

Tabla 1: Parámetros de configuración del generador LCG en Python y Java.

Parámetro	Valor	Función
Multiplicador (a)	1664525	Factor lineal; $(a-1)$ es divisible por 2 y 4.
Incremento (c)	1013904223	Constante impar; cumple $\gcd(c, 2^{32}) = 1$.
Módulo (m)	$2^{32} = 4294967296$	Límite modular y período máximo teórico.
Semilla base (X_0)	12345	Valor inicial del Escenario Base.
Semilla alternativa (X_0')	98765	Valor inicial para comparar el cambio de semilla.
Muestra (N)	$10\,000$	Total de valores generados por escenario.
Muestra en consola	20	Primeras observaciones impresas y comparadas.

4.2. Las 7 etapas de un proyecto de simulación

- **Etapa 1. Definición del problema:** Implementar el algoritmo LCG en Python y Java para generar $10\,000$ datos, verificar que produzcan la misma secuencia y evaluar el cambio de semilla.
- **Etapa 2. Formulación conceptual:** Definir las ecuaciones de X_{n+1} y U_n, la búsqueda de ciclos empíricos con tablas hash y las reglas de Hull–Dobell.
- **Etapa 3. Recolección y análisis de datos:** Configurar $a = 1664525$, $c = 1013904223$, $m = 2^{32}$ y $N = 10\,000$ con dos semillas: 12345 (base) y 98765 (alternativa).
- **Etapa 4. Construcción del modelo:** Programar ambos proyectos bajo arquitectura MVC + Servicios (`generator`, `service`), usando `long` de 64 bits en Java para evitar desbordamientos.
- **Etapa 5. Verificación y validación:** Comprobar que las primeras 20 observaciones coincidan exactamente entre Python y Java y que los $10\,000$ valores sean únicos ($P = 2^{32}$).
- **Etapa 6. Experimentación y análisis de resultados:** Calcular las diferencias $|U_1(n) - U_2(n)|$ entre semillas y graficar los primeros 200 puntos junto con el histograma de 20 clases.

- **Etapa 7. Implementación y documentación:** Subir los proyectos `Python/` y `Java/` a GitHub y documentar los resultados en este informe.

4.3. Arquitectura MVC con capa de Servicios en Python y Java

Ambos proyectos dividen el código en cinco paquetes (`model`, `generator`, `service`, `view` y `controller`) aplicando *Clean Code* y principios **SOLID** (Tabla 2):

Tabla 2: Estructura modular en Python y Java (MVC + Generator/Service).

Módulo (Python / Java)	**Responsabilidad**
`model/model.py` `model/Model.java`	Clases inmutables: `LCGScenario`, `PeriodAnalysisResult` y `SimulationRun`.
`generator/number_generator.py` `generator/NumberGenerator.java`	Calcula $X_n = (aX_{n-1} + c) \bmod m$ y $U_n = X_n/m$ para $N = 10\,000$.
`service/analyzer.py` `service/Analyzer.java`	Detecta ciclos en $O(N)$ con tablas hash y evalúa el teorema de Hull–Dobell.
`service/graphic_generator.py` `service/GraphicGenerator.java`	Grafica los primeros 200 valores U_n y el histograma de 20 intervalos.
`view/view.py` `view/View.java`	Imprime las 20 observaciones iniciales, el período y la comparación de semillas.
`controller/controller.py` `controller/Controller.java`	Ejecuta los escenarios y conecta el generador, el analizador y las vistas.
`main.py` `Main.java`	Punto de entrada que configura las semillas `12345` y `98765`.

5. Resultados

5.1. Comparación numérica entre Python y Java y cambio de semilla

Con $a = 1664525$, $c = 1013904223$ y $m = 2^{32}$, Python y Java generan exactamente los mismos valores X_n y U_n. Las Tablas 3 y 4 resumen las primeras 10 observaciones de ambas semillas y el análisis de período.

Tabla 3: Observaciones generadas en Python y Java (Semilla 1 = 12345 vs. Semilla 2 = 98765).

n	$X_1(n)$ **(Py = Java)**	$U_1(n)$ **(Sem. 12345)**	$X_2(n)$ **(Py = Java)**	$U_2(n)$ **(Sem. 98765)**	$\lvert U_1 - U_2 \rvert$
1	87628868	0.0204026858	2201958600	0.5126834399	0.4922807541
2	71072467	0.0165478482	2731312519	0.6359332528	0.6193854046
3	2332838374	0.5431562597	135349706	0.0315135594	0.5116427003
4	1761074861	0.4100321936	1473772193	0.3431393276	0.0668928660
5	313684584	0.0730353837	972805004	0.2264988162	0.1534634325
6	1852882087	0.4314077276	3148966059	0.7331757953	0.3017680677
7	2690497306	0.6264302192	1298940174	0.3024330764	0.3239971428
8	1882405609	0.4382817096	273755805	0.0637387402	0.3745429694
9	1456332196	0.3390787626	2991532632	0.6965204673	0.3574417047
10	4226455299	0.9840483076	4240292791	0.9872701161	0.0032218085

Tabla 4: Análisis de período empírico y teórico (Hull–Dobell) en Python y Java.

Métrica	**Base** (12345)	**Alternativa** (98765)
Muestra evaluada (N)	$10\,000$ valores	$10\,000$ valores
Valores únicos	$10\,000$ ($100\,\%$)	$10\,000$ ($100\,\%$)
Ciclo empírico en muestra	Sin repeticiones	Sin repeticiones
Cumple Hull–Dobell	Sí ($\gcd = 1$, $p = 2$, div 4)	Sí ($\gcd = 1$, $p = 2$, div 4)
Período teórico (P)	$m = 4\,294\,967\,296$ (2^{32})	$m = 4\,294\,967\,296$ (2^{32})

5.2. Salidas en consola divididas (Python y Java)

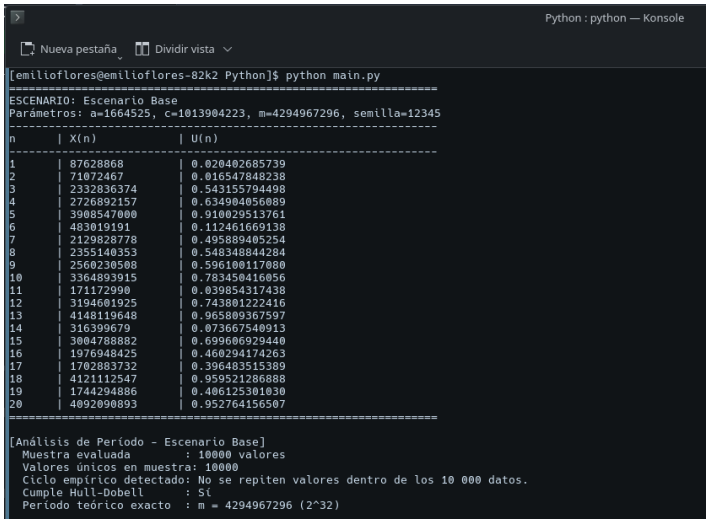


Figura 1: Salida en consola en Python (Parte 1/3)

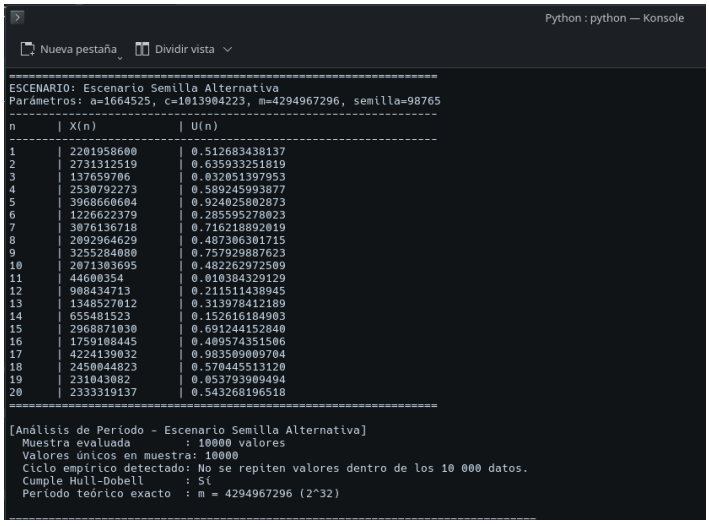


Figura 2: Salida en consola en Python (Parte 2/3)

```
================================================================================
COMPARACIÓN DE LAS PRIMERAS 20 OBSERVACIONES (Semilla 1 = 12345 vs Semilla 2 = 98765)
--------------------------------------------------------------------------------
n     | X1(n)         | U1(n)          || X2(n)         | U2(n)          | |U1 - U2|
--------------------------------------------------------------------------------
1     | 87628868      | 0.0204026857   || 2201958600     | 0.5126834381   | 0.4922807524
2     | 71072467      | 0.0165478482   || 2731312519     | 0.6359332518   | 0.6193854036
3     | 2332836374    | 0.5431557945   || 137659706      | 0.0320513980   | 0.5111043965
4     | 2726892157    | 0.6349040561   || 2530792273     | 0.5892459939   | 0.0456580622
5     | 3908547000    | 0.9100295138   || 3968660604     | 0.9240258029   | 0.0139962891
6     | 483019191     | 0.1124616691   || 1226622379     | 0.2855952780   | 0.1731336089
7     | 2129828778    | 0.4958894053   || 3076136718     | 0.7162188920   | 0.2203294868
8     | 2355140353    | 0.5483488443   || 2092964629     | 0.4873063017   | 0.0610425426
9     | 2560230508    | 0.5961001171   || 3255284080     | 0.7579298876   | 0.1618297705
10    | 3364893915    | 0.7834504161   || 2071303695     | 0.4822629725   | 0.3011874435
11    | 171172990     | 0.0398543174   || 44600354       | 0.0103843291   | 0.0294699883
12    | 3194601925    | 0.7438012224   || 908434713      | 0.2115114389   | 0.5322897835
13    | 4148119648    | 0.9658093676   || 1348527012     | 0.3139784122   | 0.6518309554
14    | 316399679     | 0.0736675409   || 655481523      | 0.1526161849   | 0.0789486440
15    | 3004788882    | 0.6996069294   || 2968871030     | 0.6912441528   | 0.0083627766
16    | 1976948425    | 0.4602941743   || 1759108445     | 0.4095743515   | 0.0507198228
17    | 1702883732    | 0.3964835154   || 4224139032     | 0.9835090097   | 0.5870254943
18    | 4121112547    | 0.9595212869   || 2450044823     | 0.5704455131   | 0.3890757738
19    | 1744294886    | 0.4061253010   || 231043082      | 0.0537939095   | 0.3523313915
20    | 4092090893    | 0.9527641565   || 2333319137     | 0.5432681965   | 0.4094959600
================================================================================
```

Figura 3: Salida en consola en Python (Parte 3/3)

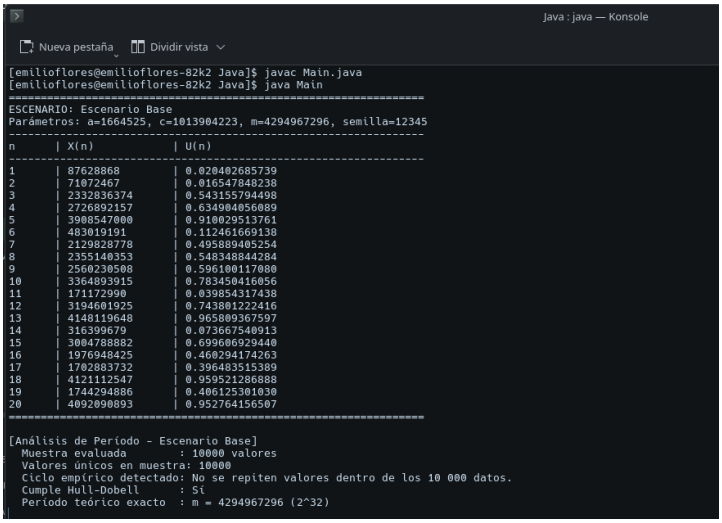


Figura 4: Salida en consola en Java (Parte 1/3)

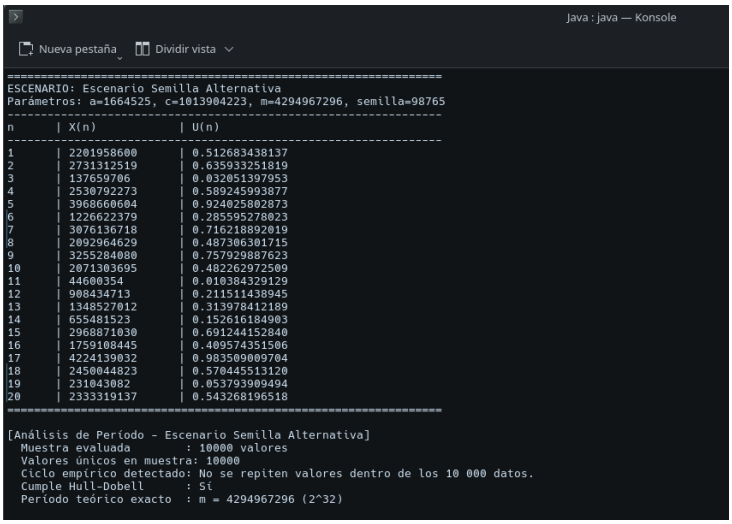


Figura 5: Salida en consola en Java (Parte 2/3)

```
================================================================================
COMPARACIÓN DE LAS PRIMERAS 20 OBSERVACIONES (Semilla 1 = 12345 vs Semilla 2 = 98765)
--------------------------------------------------------------------------------
n     | X1(n)         | U1(n)          || X2(n)         | U2(n)          | |U1 - U2|
--------------------------------------------------------------------------------
1     | 87628868      | 0.0204026857   || 2201958600   | 0.5126834381   | 0.4922807524
2     | 71072467      | 0.0165478482   || 2731312519   | 0.6359332518   | 0.6193854036
3     | 2332836374    | 0.5431557945   || 137659706    | 0.0320513980   | 0.5111043965
4     | 2726892157    | 0.6349040561   || 2530792273   | 0.5892459939   | 0.0456580622
5     | 3908547000    | 0.9100295138   || 3968660604   | 0.9240258029   | 0.0139962891
6     | 483019191     | 0.1124616691   || 1226622379   | 0.2855952780   | 0.1731336089
7     | 2129828778    | 0.4958894053   || 3076136718   | 0.7162188920   | 0.2203294868
8     | 2355140353    | 0.5483488443   || 2092964629   | 0.4873063017   | 0.0610425426
9     | 2560230508    | 0.5961001171   || 3255284080   | 0.7579298876   | 0.1618297705
10    | 3364893915    | 0.7834504161   || 2071303695   | 0.4822629725   | 0.3011874435
11    | 171172990     | 0.0398543174   || 44600354     | 0.0103843291   | 0.0294699883
12    | 3194601925    | 0.7438012224   || 908434713    | 0.2115114389   | 0.5322897835
13    | 4148119648    | 0.9658093676   || 1348527012   | 0.3139784122   | 0.6518309554
14    | 316399679     | 0.0736675409   || 655481523    | 0.1526161849   | 0.0789486440
15    | 3004788882    | 0.6996069294   || 2968871030   | 0.6912441528   | 0.0083627766
16    | 1976948425    | 0.4602941743   || 1759108445   | 0.4095743515   | 0.0507198228
17    | 1702883732    | 0.3964835154   || 4224139032   | 0.9835090097   | 0.5870254943
18    | 4121112547    | 0.9595212869   || 2450044823   | 0.5704455131   | 0.3890757738
19    | 1744294886    | 0.4061253010   || 231043082    | 0.0537939095   | 0.3523313915
20    | 4092090893    | 0.9527641565   || 2333319137   | 0.5432681965   | 0.4094959600
================================================================================
```

Figura 6: Salida en consola en Java (Parte 3/3)

5.3. Gráficas de secuencia e histogramas divididas (Python y Java)

Las gráficas de `Matplotlib` (Python) y `Swing` (Java) se dividen en dos mitades: Escenario Base (`12345`) y Semilla Alternativa (`98765`).

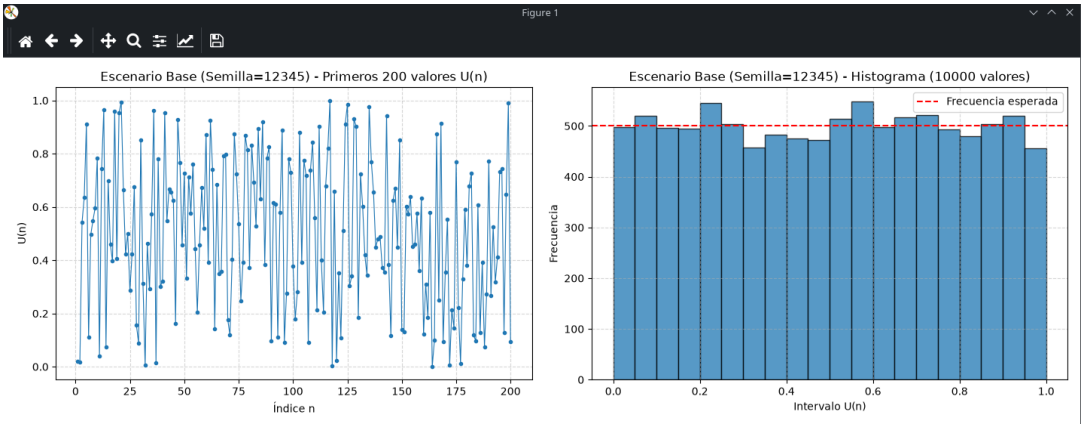


Figura 7: Gráfica en Python (Mitad 1/2 - Semilla 12345): primeros 200 valores U_n e histograma de $10\,000$ datos.

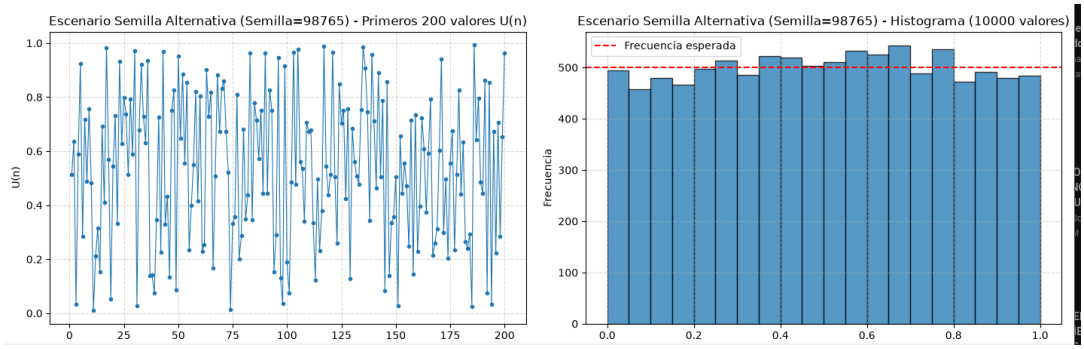


Figura 8: Gráfica en Python (Mitad 2/2 - Semilla 98765): primeros 200 valores U_n e histograma de $10\,000$ datos.

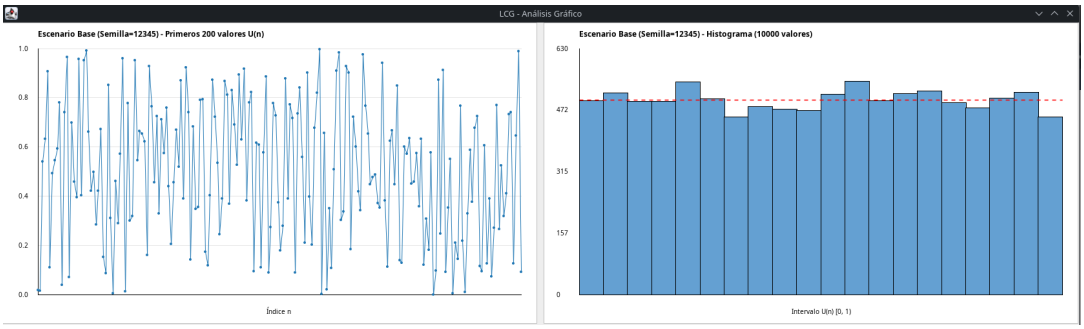


Figura 9: Gráfica en Java Swing (Mitad 1/2 - Semilla 12345): primeros 200 valores U_n e histograma de $10\,000$ datos.

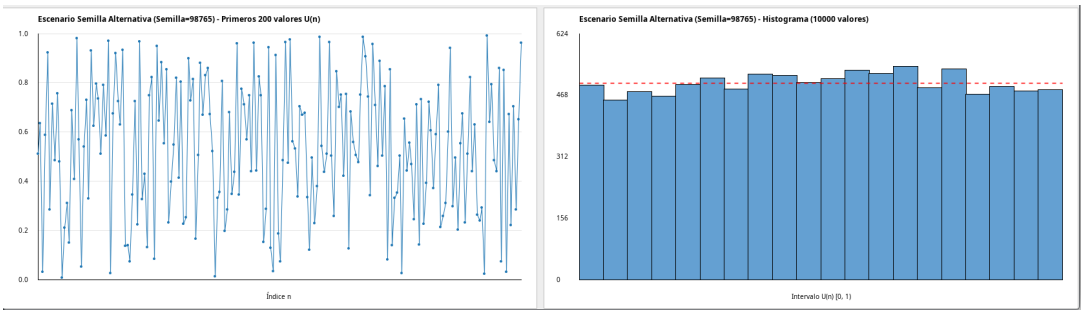


Figura 10: Gráfica en Java Swing (Mitad 2/2 - Semilla 98765): primeros 200 valores U_n e histograma de $10\,000$ datos.

5.4. Análisis de resultados

- **Igualdad entre Python y Java:** Ambos lenguajes entregan exactamente los mismos enteros X_n y decimales U_n (Tabla 3), confirmando que usar variables `long` de 64 bits en Java evita el desbordamiento en $m = 2^{32}$.
- **Uniformidad en las gráficas:** Los primeros 200 puntos oscilan en $[0, 1)$ sin patrones visibles, y los histogramas de 20 intervalos se mantienen próximos a la frecuencia esperada de 500 datos por clase.
- **Sensibilidad a la semilla:** Cambiar X_0 de 12345 a 98765 modifica toda la secuencia desde $n = 1$ ($U_1 = 0.0204$ vs. 0.5127), pero conserva el período completo $P = 2^{32}$ al depender solo de a, c y m.

5.5. Cumplimiento de resultados esperados

Tabla 5: Verificación de resultados esperados en la práctica APE 005.

Resultado esperado	**Cumplimiento**
LCG en Python ($10\,000$ datos).	Implementado en `Python/` para las semillas `12345` y 98765.
LCG equivalente en Java.	Implementado en `Java/` con tipo `long`, idéntico a Python.
Comparación de 20 observaciones.	Impresa en consola y resumida en la Tabla 3.
Gráficas y análisis del período.	Gráficas de 200 puntos, histogramas y prueba de Hull–Dobell ($P = 2^{32}$).
Cambio y comparación de semilla.	Evaluado entre el Escenario Base (`12345`) y Alternativo (98765).

6. Preguntas de Control

1. **¿Qué significa pseudoaleatorio?**
   Que los números se calculan con una ecuación determinista ($X_{n+1} = (aX_n + c) \bmod m$), pero estadísticamente se comportan como variables uniformes e independientes en $[0, 1)$ [1, 3].
2. **¿Qué función cumple la semilla?**
   Es el valor inicial (X_0) del generador; mantenerla fija permite repetir el mismo experimento y cambiarla genera secuencias distintas para nuevas réplicas [1, 4].
3. **¿Qué significa período?**
   Es la cantidad de números únicos que produce el generador antes de repetir un valor y ciclar la secuencia (máximo $P = m$ en un LCG) [2, 3].
4. **¿Qué ocurre si se utilizan parámetros inadecuados?**
   Si a, c y m incumplen Hull–Dobell, el ciclo se vuelve corto, los valores se repiten pronto y aparecen correlaciones o huecos en el intervalo $[0, 1)$ [3, 4].
5. **¿Qué ventaja tiene implementar el mismo algoritmo en dos lenguajes?**
   Permite validar que no existan errores de redondeo ni desbordamiento de enteros (*overflow*), asegurando que la simulación sea reproducible en cualquier plataforma [1, 4].

7. Conclusiones

- Las implementaciones del LCG en Python y Java bajo arquitectura MVC generaron exactamente la misma secuencia de $10\,000$ valores enteros X_n y normalizados U_n.
- El módulo `Analyzer` verificó el cumplimiento del teorema de Hull–Dobell ($P = 2^{32}$), obteniendo $10\,000$ datos únicos sin ciclos empíricos y una distribución uniforme alrededor de 500 datos por intervalo.
- El cambio de semilla (`12345` vs. `98765`) generó una trayectoria distinta desde el primer paso, manteniendo la uniformidad y el período máximo del generador.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 005: Implementación y análisis del Generador Congruencial Lineal en Python y Java*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 5: Generación de Números Pseudoaleatorios y Método Congruencial Lineal*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. T. E. Hull y A. R. Dobell, “Random number generators”, *SIAM Review*, vol. 4, no. 3, pp. 230–254, 1962.
4. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
5. D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3.ª ed. Reading, MA, EE. UU.: Addison-Wesley, 1997.

9. Anexos

Anexo A. Instalación, compilación y ejecución en Python y Java

Repositorio del proyecto: Repositorio GitHub

```
# 1. Clonar el repositorio
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git

# 2. Ejecutar la implementacion en Python
cd Repositorio-Simulacion/Ape5/Python
python3 -m venv .venv
source .venv/bin/activate
pip install matplotlib
python main.py

# 3. Compilar y ejecutar la implementacion en Java
cd ../Java
javac Main.java
java Main
```

Anexo B. Declaración de uso de inteligencia artificial

Se declara el uso de herramientas de inteligencia artificial generativa como apoyo para estructurar la arquitectura MVC en Python y Java, programar la validación de Hull–Dobell y generar las vistas gráficas.

Prompt utilizado para el desarrollo de la arquitectura y diseño del código:

> *"Partiendo del contexto teórico sobre "Generador Congruencial Lineal (LCG)" con parámetros* $a = 1664525$, $c = 1013904223$, $m = 2^{32}$, $N = 10000$ *y semillas* 12345 *y* 98765*, implementa el mismo algoritmo tanto en Python como en Java aplicando una arquitectura Modelo-Vista-Controlador (MVC) extendida con capas de generación (`generator`) y servicios (`service`), respetando normas de Clean Code y principios SOLID. En el modelo (`LCGScenario`, `PeriodAnalysisResult`, `SimulationRun`) usa estructuras de datos inmutables; en `NumberGenerator` calcula* $X(n+1) = (aX(n) + c) \bmod m$ *y* $U(n) = X(n)/m$ *cuidando usar enteros de 64 bits (`long`) en Java para evitar desbordamientos; en `Analyzer` detecta ciclos empíricos con tablas hash en* $O(N)$ *y verifica el teorema de Hull-Dobell mediante* $\gcd$ *y factores primos; en `View` imprime las primeras 20 observaciones, el análisis de período y la comparación entre semillas; y en `GraphicGenerator` grafica los primeros 200 valores y el histograma de 20 intervalos usando Matplotlib en Python y Swing/Graphics2D en Java."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)

- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)
- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.